

Time-Locked Attestations

Commit-Reveal as a Runtime Primitive

Ligate Labs Research, Working Paper v0.2.1

Date: 2026-05-28

Contact: hello@ligate.io

Abstract

A standard attestation publishes its payload at submission time. For some workloads this is wrong: a sealed-bid auction wants commitments now and reveals after close; an embargoed announcement attests now and goes public at announcement time; a regulatory filing must land at deadline but become public only after a retention period. Off-chain commit-reveal protocols handle these cases at the cost of trusting an intermediary to hold the sealed payload until reveal time.

This paper specifies **time-locked attestations** as a runtime primitive. A schema field `reveal_at` declares the block height after which the payload becomes valid for reveal. The commit phase submits a binding commitment $h = H(\text{payload} \parallel \text{nonce})$ as the attestation. The reveal phase submits $(\text{payload}, \text{nonce})$, which the runtime checks against h and the time-bound. Validators enforce the reveal-window; never-revealed commitments age out via TTL with explicit cleanup semantics.

The mechanism is positioned as a **v1.5 protocol feature**. v1 of Ligate Chain ships single-phase attestations only. v0.2 of this paper begins authoring when at least one design partner per use-case category has submitted a concrete use case.

Three contributions. First, we specify the protocol-level mechanism (§4): `MsgCommit`, `MsgReveal`, `MsgCleanup`, the commitment lifecycle state machine, the optional deposit-on-commit, and the front-running defense via batched-reveal sequencing. Second, we analyze cryptographic soundness (§5): the commit-reveal scheme is binding under collision-resistance of the chosen hash (SHA-256 default) and hiding under sufficient nonce entropy (128-bit minimum); the time-lock semantics are enforced by validators reading chain height, no exotic primitives required. Third, we analyze six failure modes (§8) with bounded damage arguments: never-reveal, late-reveal, front-running between commit and reveal, hash collisions, nonce reuse, and reveal-DoS.

The mechanism is positioned as a **v1.5 protocol feature**, post-devnet but pre-mainnet. v1 ships with single-step attestations only; time-locked attestations land when at least one design-partner use case per category (auction, embargo, regulatory time-lock) is validated. §6 documents the gate. This paper exists so the design space is captured before then, not as a roadmap commitment to ship.

1. Introduction

1.1 The Late-Disclosure Thesis

A standard attestation publishes its payload at submission time. The chain knows what the attester claimed; downstream consumers read the payload immediately. For most workloads this is the right design: PoUA reputation, content provenance, identity proofs, audit trails are all “we observed X and we want everyone to know now” patterns.

A category of workloads breaks this. A bidder in a sealed-bid auction wants to commit to a bid value now and reveal it only after the auction closes; revealing early gives competitors information. A journalist embargoed on a story wants to commit to the article now and publish only at the agreed embargo time; publishing early breaches the embargo. A regulatory filer wants to submit a filing at the legal deadline but make it publicly readable only after a retention period; publishing immediately violates the regime.

These workloads share a structural feature: the attestation must exist (be recorded, be binding,

be auditable later) before the payload becomes public. The standard single-phase attestation cannot do this. Off-chain commit-reveal protocols handle these cases at the cost of trusting an intermediary (an auctioneer, a press agent, a regulator’s portal) to hold the payload until reveal time. The intermediary is a single point of failure for confidentiality, integrity, and availability.

The thesis of this paper: **for an attestation-native chain, time-locked commit-reveal should be a runtime primitive, not an off-chain protocol or an application-layer pattern.** Reasons in §1.2 and §1.3.

1.2 The Off-Chain Commit-Reveal Trust Problem

Standard off-chain commit-reveal: a bidder publishes $H(\text{bid} \parallel \text{nonce})$ to some channel (often a chain), then later sends $(\text{bid}, \text{nonce})$ to an auctioneer at close. The auctioneer collects reveals, tallies, declares a winner.

This requires trusting the auctioneer to:

- Actually run the auction (availability)
- Not reveal payloads early to favored bidders (confidentiality)
- Tally honestly (integrity)
- Not censor reveals from disfavored bidders (censorship-resistance)

Each is a real failure mode in production auctions. Auctioneers go offline; auctioneers leak; auctioneers cheat; auctioneers censor. Modern auction-design literature treats auctioneer-honesty as a thick assumption that needs separate mechanism to harden.

On-chain commit-reveal removes the auctioneer entirely. The chain holds the commitment. The chain validates the reveal against the commitment. Validators provide availability (BFT chain liveness); the chain provides integrity (consensus on which reveals landed); censorship-resistance is the chain’s standard property. The trust assumption shifts from “the auctioneer is honest” to “the chain consensus holds,” which is a much weaker and more auditable assumption.

The same argument applies to embargoed announcements (replace “auctioneer” with “embargo coordinator”) and regulatory time-locks (replace “auctioneer” with “regulator’s filing portal”). In each case, an on-chain primitive replaces a trusted intermediary with chain consensus.

1.3 Why Now (or Why v1.5, Not v1)

Ligate Chain v1 ships with single-phase attestations. The four flagship products (Themisra, Mneme, Iris, Atlas) do not need commit-reveal at launch. v1 engineering cost is bounded by the minimum surface needed to ship.

Time-locked attestations matter when use cases like sealed-bid auctions, embargoed press releases, and regulatory filings ramp up. The v1.5 timing reflects this: post-devnet, post-Mneme-launch, when at least one design partner per category (auction, embargo, regulatory) has submitted a concrete use-case description matching the §6.2 template. The §6.1 gate enforces this discipline.

The primitive is small. The §4 mechanism is three transaction types (`MsgCommit`, `MsgReveal`, `MsgCleanup`) and a four-state lifecycle. The §5 cryptographic security argument is one page. The total runtime surface is comparable to native delegation: cleanly addable post-launch without disruption to existing schemas.

This paper does not advocate for shipping time-locked attestations on v1 day one. It documents the design so when demand validates, engineering has a target.

1.4 The Central Question

What is the minimum on-chain commit-reveal primitive that handles the three use-case families (sealed-bid auction, embargoed announcement, regulatory time-lock), with adequate cryptographic security and explicit never-reveal cleanup, without re-introducing the trust assumptions of off-chain protocols?

This paper answers: `MsgCommit` + `MsgReveal` + `MsgCleanup` over a four-state lifecycle (`COMMITTED` → `REVEALED` or `EXPIRED` → `CLEANED-UP`), with hash-based commitments under collision-resistant H (default SHA-256, 128-bit nonce minimum), time-bounds enforced by validators reading chain height, and a configurable deposit-on-commit + cleanup-runner reward to fund the cleanup market.

1.5 Approach in Brief

Each schema may declare a `reveal_at` field on its commitments. `MsgCommit` (§4.1) publishes $h = H(\text{payload} \parallel \text{nonce})$ along with the schema’s chosen `reveal_at` and `ttl`. The chain records the commitment in state.

After `reveal_at` (and within `ttl`), the schema’s attester set submits `MsgReveal` (§4.2) with $(\text{payload}, \text{nonce})$. The runtime checks $H(\text{payload} \parallel \text{nonce}) = h$ and the time-bound; if both pass, the commitment transitions to `REVEALED` and the payload becomes the canonical attestation.

If `reveal_at` + `ttl` passes without reveal, the commitment transitions to `EXPIRED`. Anyone can submit `MsgCleanup` (§4.3) to permanently remove the `EXPIRED` commitment from active state, earning a small protocol fee from the schema’s deposit (if any). This creates a permissionless cleanup market: independent operators monitor the chain for expired commitments and submit cleanup transactions.

The mechanism composes with native delegation (a hot key with a grant scoped to schema σ can submit commits and reveals on the master’s behalf), with per-schema fees (commits and reveals pay base fee + tip per the schema’s fee curve), and with cross-schema composition (a revealed attestation can be referenced as input by downstream schemas, with the §5 cascade firing if the reveal is later invalidated).

1.6 Contributions

The paper makes four contributions.

A **mechanism specification** in §3 and §4: the commitment tuple, the validity state machine, the three transaction types, the deposit-on-commit and cleanup-runner economics, the front-running defense via batched-reveal sequencing.

A **cryptographic security analysis** in §5: binding under collision-resistance of H , hiding under pre-image-resistance and 128-bit nonce floor, time-lock security under chain consensus. The arguments are standard but documented explicitly so an implementer or security reviewer has a single source.

A **failure-mode analysis** in §8: six failure modes (never-reveal, late-reveal, front-running between commit and reveal, hash collisions, nonce reuse, reveal-DoS) with bounded-damage arguments for each.

A **use-case validation gate** in §6: explicit framing that this paper is specification-only until 2-3 design partners across categories submit concrete use-case descriptions matching the §6.2 template.

1.6.1 Status of Claims Proven (formal mathematical argument under standard cryptographic assumptions):

- §5.1 binding: under SHA-256 collision-resistance, no committer can produce two valid reveals for the same commitment with different payloads. Standard hash-commitment argument; documented for completeness.
- §5.2 hiding: under SHA-256 pre-image-resistance and 128-bit nonce floor, the payload cannot be recovered from the commitment without exhaustive enumeration.
- §5.4 time-lock security: under chain BFT liveness, reveal-before-reveal_at is unambiguously rejected; reveal-after-reveal_at + ttl is unambiguously rejected.

Bounded under stated assumptions:

- §5.1 + §5.2 assume the chosen hash function is collision-and-pre-image-resistant at the 128-bit level. If a future cryptanalytic break weakens SHA-256, schemas must migrate via §3.4's hash-function-migration path.
- §8.3 front-running defense assumes the §4.5 batched-reveal sequencing is enforced; an architecture that separates proposer from builder requires re-validation.
- §8.6 reveal-DoS bound assumes the cleanup-runner economics (§4.3) actually attract independent operators; if no operator submits cleanup, expired commitments accumulate in state.

Empirical or heuristic, requiring devnet validation:

- §6 use-case fit: not a chain claim; a process claim. The mechanism is specified; whether real workloads need it depends on design-partner submission.
- §3.4 nonce-length recommendation: 128-bit is the conservative floor; real-world workloads may inform whether 256 bits is needed for some payload-space distributions.

1.7 Scope and Non-Goals

In scope:

- On-chain commit-reveal as a runtime primitive
- Three transaction types (commit, reveal, cleanup) and the four-state lifecycle
- Deposit-on-commit and cleanup-runner economics
- Front-running defense via batched-reveal sequencing
- Cryptographic security under standard hash-function assumptions
- Six failure-mode bounds

Explicitly out of scope:

- **Zero-knowledge variants.** A reveal that proves “the committed payload satisfies predicate P” without disclosing the payload is a useful extension; v0.2 specifies the cleartext-reveal mechanism. ZK extension lives in a follow-up paper (§9.1).

- **Vickrey-specific auction protocols.** Vickrey, Vickrey-Clarke-Groves, and other auction protocols build on top of the commit-reveal primitive but are application-layer; this paper specifies the primitive.
- **Cross-chain time-locks.** A commitment on Ligate Chain that should reveal at a particular height on Ethereum (or any counterparty chain) is the cross-chain composition problem; out of scope.
- **Time-lock encryption.** Drand timelock encryption / VDF-based time-locks complement commit-reveal at a different point in the design space (§7); not addressed here.
- **General privacy-preserving attestation.** Time-locked attestation provides confidentiality until reveal; permanent confidentiality (zero-knowledge attestation) is a different problem.

1.8 Document Structure

Section 1.6.1 separates the paper’s claims into proven, bounded-under-stated-assumptions, and empirical-or-heuristic. Section 2 surveys cryptographic commitments, on-chain auctions, off-chain commit-reveal protocols, time-locked encryption, and embargoed-document storage networks as background. Section 3 fixes the system model: commitment, reveal, validity state machine, hash function choice. Section 4 specifies the mechanism: commit / reveal / cleanup transactions, deposit, front-running defense. Section 5 analyzes cryptographic security. Section 6 documents the use-case validation gate. Section 7 positions time-locked attestations against prior systems. Section 8 analyzes six failure modes. Section 9 lists limitations and future work. Section 10 concludes.

2. Background and Related Work

This section surveys five families of related work, each illuminating a different facet of the design space.

2.1 Cryptographic Commitments

A **commitment scheme** is a cryptographic protocol where a committer publishes a binding fingerprint of a value (the commitment), then later reveals the value with a proof that it matches the commitment. The two properties commitment schemes must have are **binding** (the committer cannot change the value after commit) and **hiding** (the verifier cannot learn the value from the commitment alone).

Three families dominate:

- **Hash-based commitments.** Compute $h = H(\text{payload} \parallel \text{nonce})$. Binding under collision-resistance of H ; hiding under pre-image-resistance of H plus nonce entropy. Used in Bitcoin Lightning, blockchain commit-reveal auctions, and countless off-chain protocols. Simple to implement; cheap to verify; bandwidth-efficient.
- **Pedersen commitments.** Compute $h = g^r \cdot k^v$ in a discrete-log group. Binding under discrete-log hardness; hiding unconditional (statistical). Has the homomorphic property that $\text{Commit}(v_1) \cdot \text{Commit}(v_2) = \text{Commit}(v_1 + v_2)$. Used in confidential transactions (Monero ringCT, Bulletproofs) where homomorphic aggregation matters.
- **Lamport one-way commitments.** Use a sequence of one-way function outputs. Post-quantum-resistant (relies on hash-function pre-image-resistance only, not on discrete-log hardness). Bandwidth-heavy: per-bit commitment overhead is significant.

What this paper picks. Hash-based commitments. Three reasons: (1) simplest implementation, smallest commit-transaction size, fastest verification; (2) collision-resistance and pre-image-resistance assumptions are well-understood for SHA-256 and BLAKE3; (3) the homomorphic property of Pedersen commitments is not needed for the three use-case families this paper targets (auctions, embargos, regulatory filings).

Future work (§9.1) covers the ZK-friendly path via Poseidon, which is hash-based but enables zero-knowledge reveal extensions.

2.2 On-Chain Auctions

Vickrey auction (second-price sealed-bid): bidders commit bids, reveal at close, highest bidder wins and pays the second-highest bid. Vickrey’s truth-telling property (it is dominant strategy to bid your true value) makes it the gold standard for sealed-bid auctions in mechanism-design theory. On Ethereum, Vickrey has been implemented as smart contracts with off-chain or on-chain auctioneers.

English ascending auction: bidders publicly bid; the auction ends when no one outbids. Does not need commit-reveal; on-chain implementations are straightforward but lose the truth-telling property.

Vickrey-Clarke-Groves (VCG) mechanisms: generalize Vickrey to multi-item auctions. Require commit-reveal per item plus a global tally; complexity grows quickly.

What this paper offers that on-chain auction contracts do not. Per-contract auction implementations re-encode the commit-reveal mechanics in Solidity. Each contract is audited separately. Adversaries exploit subtle differences in commit-reveal implementations (timing, deposit handling, cleanup) across contracts. This paper’s runtime primitive standardizes the mechanics: every schema gets the same chain-enforced commit-reveal, every auditor reviews the same protocol-level argument.

2.3 Off-Chain Commit-Reveal Protocols

RANDAO (Ethereum’s beacon-chain randomness): validators commit hashes of their entropy contributions, then reveal at the end of the epoch. The reveal-window is enforced by the beacon chain. RANDAO’s failure mode is last-revealer manipulation: a validator who is the last to reveal can choose whether to reveal (committing the entropy) or withhold (forcing a fallback that may or may not include their contribution).

Verifiable Random Functions (VRFs): a signer publishes a value $f(x)$ that they prove was computed deterministically from a secret key and a public input x . After commit-reveal of x , anyone can verify $f(x)$. Used in many on-chain random-beacon and committee-selection protocols.

Drand: a distributed-randomness beacon with t-of-n threshold signing. Each round produces a verifiable random output. Drand’s “timelock encryption” mode lets a sender encrypt a message such that it can only be decrypted after a specific round; this is a different design (cryptographic time-lock, not commit-reveal time-lock) that complements this paper’s mechanism.

What this paper takes. The RANDAO last-revealer-manipulation lesson (§4.5 front-running defense via batched-reveal sequencing prevents the analog attack). The VRF-determinism insight (the §4.4 deposit-on-commit gives an economic incentive to actually reveal). The Drand timelock encryption pattern (named in §7 as a complement, not a competitor).

2.4 Time-Locked Encryption

Drand timelock encryption (the `quicknet` mode launched in 2023): a sender encrypts a message under the threshold-signature key for a future Drand round. Only the Drand network’s threshold signature for that round can decrypt it. The receiver can decrypt at or after the target round; nobody (including the sender) can decrypt earlier.

VDF-based time-locks (Wesolowski VDF, Pietrzak VDF): a verifiable delay function takes T units of sequential computation to evaluate but is verifiable in $O(\log T)$. The sender commits a value; the verifier evaluates the VDF for T steps to recover the value. Time-lock is enforced by the inherent serial-compute requirement; no trusted beacon needed.

Proof-of-elapsed-time (Intel SGX): a TEE attests that a certain amount of time has elapsed since some event. Used in Intel’s PoET consensus and some niche time-lock protocols.

What this paper takes. Each of these is a different design point. Drand timelock relies on a trusted beacon network. VDFs rely on sequential-compute assumptions. SGX relies on hardware-attestation trust. This paper’s chain-consensus time-lock is in a fourth category: enforced by the chain’s BFT consensus reading chain height, no exotic cryptography, no trusted beacon. The §7 comparison table positions all four.

2.5 Embargoed-Document Storage Networks

Filecoin retrieval markets: a content provider stores data on Filecoin and configures retrieval to be available only after a certain time. The storage layer enforces the embargo (storage providers refuse to serve before the embargo time). Different problem (storage with delayed retrieval, not attestation with delayed disclosure); related in problem space.

Arweave with retrieval delays: similar pattern. Arweave’s permanent storage plus retrieval-policy layer can enforce embargoed disclosure.

IPFS with encryption: a content provider encrypts data, distributes the ciphertext via IPFS, releases the key at the embargo time. Off-chain trust (the publisher controls the key release).

Why these are different from this paper. Each addresses **embargoed retrieval of stored content**. This paper addresses **embargoed disclosure of an attestation**. The two compose: a long-document attestation could include a content-hash whose underlying document is stored on Filecoin/Arweave with retrieval delay matching the commit-reveal `reveal_at`. The composition is application-layer; this paper does not specify it but acknowledges the adjacent design space.

3. System Model

This section formalizes the commit-reveal model: commitments, reveals, the commitment lifecycle, and the cryptographic primitives the chain leans on. Standard PoUA assumptions (BFT consensus, threshold-signed attester sets, deterministic block heights) carry over; this section names only what’s new for time-locked semantics.

3.1 Commitment

A **commitment** is a tuple

$$c = (h, \text{reveal_at}, \text{ttl}, \sigma, \mathcal{A}_\sigma, d)$$

where:

- $h \in \{0, 1\}^{256}$ is the **commitment hash**, $h = H(\text{payload} \parallel \text{nonce})$ under the schema's declared hash function H (default SHA-256).
- $\text{reveal_at} \in \mathbb{N}$ is the earliest block height at which the reveal becomes valid. Must be strictly greater than the commitment's inclusion height.
- $\text{ttl} \in [\text{ttl}_{\min}, \text{ttl}_{\max}]$ is the **time-to-live**: the number of blocks after reveal_at during which a valid reveal can land. Past $\text{reveal_at} + \text{ttl}$, the commitment expires.
- σ is the schema-id; the commitment is bound to the schema's payload type and validation rules.
- $\mathcal{A}_\sigma$ is the schema's attester set, identifying the threshold-signature group authorized to submit this commitment.
- $d \in \mathbb{R}_{\geq 0}$ is the optional deposit held in escrow until reveal or cleanup (§4.4).

The commitment is published on-chain at commit-time in a `MsgCommit` transaction (§4.1). Once included, it persists in chain state under the schema's commitment table, indexed by canonical `commitment_id` (a hash of the commitment fields plus the inclusion height for uniqueness).

Why time-bound is part of the commitment. A schema can support multiple in-flight commitments with different reveal windows; the commitment binds its own time-lock at commit-time. This is the runtime-primitive design: time-bounds are state, not application logic, so the chain enforces them uniformly.

3.2 Reveal

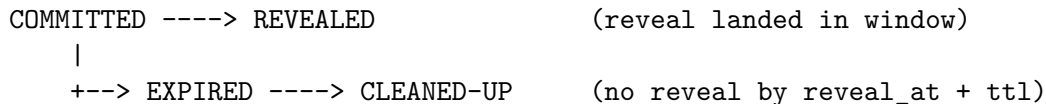
A **reveal** is the pair (payload, nonce) submitted in a `MsgReveal` transaction (§4.2), at any block height $h \in [\text{reveal_at}, \text{reveal_at} + \text{ttl})$. The runtime checks $H(\text{payload} \parallel \text{nonce}) = \text{commitment's } h$. If the check passes, the commitment transitions `COMMITTED` $\rightarrow$ `REVEALED`, the payload becomes the canonical attestation payload (readable by anyone), and any escrowed deposit returns to the committer.

Reveal must come from the same signer as the commit. The reveal carries the same threshold-signature from $\mathcal{A}_\sigma$ as the commit. This prevents a witness who learns the payload off-chain from submitting the reveal first to claim the credit; only the attester set that committed can reveal.

The reveal is the attestation. Until reveal, only the commitment exists in chain state. The chain does not (yet) know the payload. After reveal, the payload becomes the canonical attestation, indexed under the schema and visible to all readers. From an attestation-graph perspective (cross-schema-composition v0.2 §3.3), the reveal is the moment the schema's downstream consumers see the attestation as valid input.

3.3 Validity State Machine

A commitment's state evolves through a four-state machine:



State transitions:

- **COMMITTED**: initial state on inclusion of a successful `MsgCommit`.
- **COMMITTED** → **REVEALED**: fired by a successful `MsgReveal` in the reveal window. Terminal. Payload becomes canonical; deposit returns.
- **COMMITTED** → **EXPIRED**: deterministic transition at height $h \geq \text{reveal_at} + \text{ttl}$. The chain does not require an explicit transaction; expiry is computed from chain state at read time. No payload is published; the commitment “ages out.”
- **EXPIRED** → **CLEANED-UP**: fired by a `MsgCleanup` transaction (§4.3). Removes the commitment from active state, distributing the deposit per the schema’s `deposit_destination`. Terminal. A tombstone is retained for query consistency.

Cleanup is permissionless. Anyone can submit `MsgCleanup` for an **EXPIRED** commitment; the cleanup-runner earns a small protocol fee (§4.3). This creates a permissionless cleanup market: independent operators monitor the chain for expired commitments and submit cleanup transactions. Validators have priority but no monopoly.

Read-time semantics. A read of commitment c at height h returns:

- `state` = **COMMITTED** if $h < \text{reveal_at} + \text{ttl}$ and no reveal landed
- `state` = **REVEALED** plus the payload if reveal landed
- `state` = **EXPIRED** if $h \geq \text{reveal_at} + \text{ttl}$ and no reveal landed
- `state` = **CLEANED-UP** plus tombstone metadata if cleanup ran

The read-time computation is $O(1)$: look up the commitment record, compare current height against `reveal_at + ttl`. No graph traversal.

3.4 Hash Function and Nonce Length

The commitment scheme requires a **collision-resistant** and **pre-image-resistant** hash function. The protocol defaults to SHA-256; schemas may declare alternatives at registration:

- **SHA-256** (default): 256-bit output, post-quantum collision resistance estimated at 128 bits. Universally supported.
- **BLAKE3**: faster on modern hardware, same 256-bit output, same security level. Recommended for high-throughput auction schemas.
- **Poseidon**: ZK-friendly hash. Slower verification but enables zero-knowledge reveal variants (§9.1).

Nonce length minimum: 128 bits. A nonce shorter than 128 bits leaks information about low-entropy payloads (e.g., binary-decision auctions where the payload is one of two values; a 64-bit nonce can be brute-forced offline against the commitment hash). 128 bits is the protocol-level floor; schemas can require longer (256-bit nonces for sensitive embargo schemas, etc.).

Hash function migration. A schema’s hash function is set at registration and immutable. To switch (e.g., when SHA-256 falls to a future quantum attack), the schema must register a new schema-id with the new hash; existing commitments under the old schema continue under the old hash. This is the same migration pattern as native-delegation v0.2 §10.4 (PQ signature scheme upgrade): backward-compatible through schema versioning, never retroactive.

Why not require deterministic randomness from the chain? A schema author might want the chain to supply the nonce (e.g., from the block-hash beacon) to remove the committer’s

freedom in choosing nonces. The chain *could* offer this, but doing so risks committer-validator collusion (a validator who knows the chain-supplied nonce ahead of time can pre-compute possible commitments). v0 keeps the nonce committer-supplied with the 128-bit minimum; future work could explore chain-supplied-nonce variants under explicit threat models.

4. Mechanism

This section specifies the three transaction types (`MsgCommit`, `MsgReveal`, `MsgCleanup`), the commitment lifecycle state machine, the optional deposit-on-commit mechanism, and the front-running defense via batched-reveal sequencing.

4.1 Commit Transaction

The **commit transaction** publishes a binding commitment to the chain without revealing the underlying payload.

Schema.

```
MsgCommit = {
  schema_id:      SchemaId,      // bound schema
  signer:         AttestorSetSig, // threshold sig
  commitment_hash: Bytes32,      // h = H(payload || nonce)
  reveal_at:      BlockHeight,   // earliest reveal block
  ttl_blocks:     int,           // grace window
  deposit:        TokenAmount,   // optional escrow
  metadata_uri_hash: Bytes32,    // optional off-chain ptr
}
```

Validation at admission.

1. `schema_id` exists in Σ and the signer matches its attester set $\mathcal{A}_\sigma$.
2. `commitment_hash` is exactly 32 bytes.
3. `reveal_at` > current chain height H (cannot commit-and-reveal in the same block).
4. `ttl_blocks` $\in [\text{ttl}_{\min}, \text{ttl}_{\max}]$ where the bounds are governance-set protocol parameters (defaults: `ttl_min` = 6 blocks $\approx$ 72 seconds, `ttl_max` = 100,800 blocks $\approx$ 14 days).
5. `deposit` $\geq$ the schema's `deposit_floor` (registration-time parameter; default 0).
6. Standard PoUA fee payment.

State transition. On success: a new entry is appended to the chain's commitment table with state COMMITTED. Per-block tally counts this for proposer reputation (PoUA §4.3) at the schema's standard fee weight.

Per-schema deposit floor. A schema may declare a non-zero `deposit_floor` at registration. Auction schemas typically set this to a meaningful fraction of the bid value; embargo schemas typically set it to zero. The deposit is held in escrow until reveal or cleanup (§4.4).

4.2 Reveal Transaction

The **reveal transaction** publishes the original payload and proves it matches a prior commitment.

Schema.

```

MsgReveal = {
  commitment_id: CommitmentId,      // canonical id of the prior MsgCommit
  payload: Bytes,                    // original payload
  nonce: Bytes,                      // nonce used in H(payload || nonce)
  signer: AttestorSetSig             // same threshold signature as the commit
}

```

Validation at admission.

1. `commitment_id` exists in the chain's commitment table.
2. The commitment's state is COMMITTED.
3. Current height $H \in [\text{reveal_at}, \text{reveal_at} + \text{ttl_blocks})$.
4. $\$H(\text{payload} \parallel \text{nonce}) = \$ \text{commitment's commitment_hash}$ where H is the schema-declared hash function (default SHA-256).
5. The signer is the same attester-set signature as the commit (prevents reveal by an unauthorized party).
6. Standard PoUA fee payment.

State transition. On success: commitment state transitions COMMITTED $\rightarrow$ REVEALED. The payload becomes the canonical attestation payload (visible to all readers, indexed by schema). Any deposit held in escrow returns to the committer's address.

Reveal-side reputation. The schema's per-attestation reputation is credited at reveal time, not commit time. This is a deliberate design choice: the commitment alone is not a complete attestation (no payload); only the reveal closes the loop. PoUA §4.3 reputation accrual fires for the reveal signer.

4.3 Cleanup of Expired Commitments

The **cleanup transaction** removes a never-revealed commitment from active chain state.

Schema.

```

MsgCleanup = {
  commitment_id: CommitmentId,      // canonical id
  cleanup_runner: Address            // recipient of cleanup-runner reward
}

```

Validation at admission.

1. `commitment_id` exists.
2. Commitment state is COMMITTED (not yet REVEALED or EXPIRED).
3. Current height $H \geq \text{reveal_at} + \text{ttl_blocks}$.

State transition. On success:

1. Commitment state transitions COMMITTED $\rightarrow$ EXPIRED $\rightarrow$ CLEANED-UP (atomic in the same block).
2. The commitment record is removed from the active commitment table (state pruning); a tombstone with (`commitment_id`, EXPIRED) is retained for query consistency.
3. Any deposit is distributed per the schema's `deposit_destination`:
 - COMMITTER: returned to the original committer (default for low-stakes embargo schemas)
 - TREASURY: routed to the chain treasury (default for auction schemas)

- **BURN**: sent to the pure-burn address per PoUA §5.5.3 (matches the τ_{burn} destination)
- **CLEANUP_RUNNER**: routed to `cleanup_runner` field (incentivizes anyone to call cleanup on expired commits)
- **Combinations**: schemas can declare a split (e.g., 50% to runner + 50% to treasury) to fund cleanup-runner economics

Cleanup-runner economics. A small protocol fee is rebated to `cleanup_runner` regardless of `deposit_destination`. This creates a permissionless cleanup market: anyone monitoring the chain for expired commitments can submit `MsgCleanup` and earn the fee. Validators have priority but no monopoly.

4.4 Deposit-on-Commit Mechanism

The `deposit` field on `MsgCommit` is optional but governed by the schema’s `deposit_floor`.

Why deposits matter. Without a deposit, an adversary can spam commitments with no economic cost. Each commitment consumes chain state until cleanup; spam attacks are bounded only by per-tx fees. With a deposit, spam attacks pay the deposit upfront and lose it on never-reveal.

Per-schema policy.

Use case	<code>deposit_floor</code>	<code>deposit_destination</code>
Sealed-bid auction	high (e.g., $\geq$ minimum bid value)	BURN or CLEANUP_RUNNER (deters spam, never returns to committer)
Embargoed announcement	zero or symbolic	COMMITTER (return on reveal; symbolic on cleanup)
Regulatory time-lock	moderate	TREASURY (deters frivolous filings)
Insurance claim	moderate	COMMITTER on reveal; TREASURY on expiry

Deposit-floor calibration. The `deposit_floor` should make the spam-cost-per-commitment exceed the cost of one `MsgCleanup` execution. At v0 chain parameters (estimated ~ 0.001 AVOW per cleanup), a deposit floor of ~ 0.01 AVOW (10x cleanup cost) is sufficient. Schemas with high-value commitments (e.g., auctions) set the floor much higher to bound the lost-deposit risk for honest committers.

Why deposits, not just fees. The base commit fee is paid even on successful reveal; it does not deter never-reveal because reveal itself doesn’t refund the fee. The deposit is the only mechanism that creates economic incentive to reveal: lose the deposit if you don’t reveal. Deposits and fees are complementary: fees fund proposer reward / burn, deposits fund cleanup runner / spam deterrence.

4.5 Front-Running Defense

A naive reveal exposes the payload in the mempool before block inclusion. An MEV-aware adversary observing the reveal in the mempool could submit a competing commitment using the now-public

payload + a fresh nonce, then race the original reveal to settlement. This is the **commit-reveal front-running attack**.

Defense: batched-reveal sequencing.

The proposer’s block-construction rule mandates that **all reveals are sequenced before any new commits in the same block**. Specifically, a block’s transaction order is:

1. All `MsgReveal` transactions (sorted by `commitment_id`; canonical order)
2. All `MsgCleanup` transactions (sorted by `commitment_id`)
3. All other transactions (`MsgCommit`, attestations, transfers, etc.)

This ordering is enforced by the runtime: a block whose `MsgCommit` precedes any `MsgReveal` is invalid. Proposers proposing a non-conforming block are slashed per PoUA §4.5.

Why this defends. A reveal in the mempool can be observed, but the adversary cannot submit a competing commitment that lands in the same block before the reveal. The reveal is sequenced first; the original commitment is finalized; any competing commit lands in a later block. By the time the adversary’s competing commit is included, the original payload is already canonical and the adversary’s commit is at best a duplicate (rejected by the schema’s `commitment_hash` uniqueness check, §4.1 admission).

Limitations.

- **Cross-block races.** The defense protects within a single block. If the reveal takes more than one block to settle (e.g., during network congestion), the adversary has multiple opportunities to compete. Mitigated by the schema declaring a tight `ttl_blocks` (forces resolution quickly) and by validators prioritizing reveals in their mempool.
- **Validator collusion.** A validator who colludes with an adversary can submit the competing commit before the reveal. PoUA §A.1 / §A.2 detectors flag schema-mix anomalies; a validator persistently delaying reveals would deviate from the chain-wide null distribution and trigger slashing. Practical defense.
- **Encrypted-mempool variants.** A future upgrade can use encrypted mempools (à la SUAVE, Shutter Network) to prevent reveal observation entirely. Out of scope for v0; tracked as future work.

Canonical ordering rule. Within the reveals-first batch, reveals are sorted ascending by reveal-at block, then by commitment-id. This is deterministic: every honest validator constructs the same block order given the same mempool, and the rule is verifiable post-hoc by any light client.



5. Cryptographic Security

This section formalizes the security properties of the commit-reveal scheme. We show that under standard cryptographic assumptions on the hash function H and nonce entropy, the scheme is binding (committer cannot change the payload after commit) and hiding (readers cannot learn the payload before reveal). The time-lock semantics are enforced by chain consensus, not by exotic cryptography (no VDFs, no time-lock encryption required).

5.1 Binding

Claim. Under collision-resistance of H , a committer cannot change the payload after commit. Given a published commitment h , no adversary can produce two different reveals $(\text{payload}_1, \text{nonce}_1)$ and $(\text{payload}_2, \text{nonce}_2)$ that both pass the reveal check $H(\cdot) = h$ with $\text{payload}_1 \neq \text{payload}_2$.

Argument. Producing two reveals that hash to the same commitment is a second-preimage attack on H . SHA-256 second-preimage resistance is $\sim 2^{256}$ (stronger than the 2^{128} collision bound). To construct a collision-by-design would require approximately 2^{128} hash computations under the birthday bound; second-preimage against a specific known hash h is harder. Both are infeasible with current or projected hardware.

Limit of the claim. Binding is conditional on H being second-preimage-resistant. If a future cryptanalytic break reduces SHA-256 below 2^{80} work, schemas using SHA-256 must migrate via §3.4. Existing commitments under the old hash inherit the weakened binding; schemas handling high-value commitments should monitor cryptanalytic developments and rotate proactively.

5.2 Hiding

Claim. Under pre-image-resistance of H and sufficient nonce entropy, a reader of the commitment cannot learn the payload before reveal. Given h , no efficient adversary (without side-channel access to the payload) can recover payload with probability significantly better than $|\text{payload-space}| \cdot 2^{-|\text{nonce}|}$.

Argument. The adversary’s best attack is exhaustive enumeration: for each candidate payload p and each candidate nonce n , compute $H(p\|n)$ and check against h . Total work: $|\text{payload-space}| \cdot 2^{|\text{nonce}|}$ hash computations.

For a binary-decision auction ($\text{payload-space} = \{\text{YES}, \text{NO}\}$) with a 128-bit nonce, the effective security is $2 \cdot 2^{128} \approx 2^{129}$ work. Infeasible.

For an embargoed news article (payload-space essentially unbounded text, but with low entropy in the headline / first paragraph), the work is bounded below by $2^{|\text{nonce}|}$. The 128-bit floor delivers 128-bit hiding regardless of payload structure.

Caveat: side channels. Hiding is broken if the adversary has any off-chain knowledge of the payload (overheard a bidder, accessed the committer’s logs). The cryptographic argument bounds attacks operating purely on the commitment hash; it does not protect against operational leakage. Application-layer guidance: committers should be operationally careful with the payload-nonce pair until reveal.

5.3 Nonce Entropy Bound

Claim. The minimum nonce length of 128 bits is the protocol-level floor. Schemas may require more.

Argument. Per §5.2, hiding security scales as $|\text{payload-space}| \cdot 2^{|\text{nonce}|}$. For a low-entropy payload space (binary, ternary, or any small discrete set), work is dominated by the nonce length. 128 bits gives 128-bit security against brute force; 64 bits would be brute-forceable on a small GPU cluster in days (2^{64} SHA-256 hashes is $\sim 10^{15}$ ops).

Schema-declared nonce minimums. A schema can declare a longer minimum at registration (e.g., 256 bits for sensitive embargo schemas). The runtime enforces the declared minimum; commits

with shorter nonces are rejected at admission.

Why not bound higher. Longer nonces increase commit-transaction size linearly. 128 bits is the smallest size delivering 128-bit security in the worst case (binary payload space). Larger nonces are governance-tunable per-schema, not protocol-mandated.

5.4 Time-Lock Security

Claim. Reveal-before-`reveal_at` is rejected by validators reading chain height. No exotic cryptography (verifiable delay functions, time-lock encryption) is required.

Argument. The §4.2 reveal admission check verifies $H_{\text{current}} \geq \text{reveal_at}$. If $H_{\text{current}} < \text{reveal_at}$, the reveal is rejected at mempool admission. Chain height is the canonical clock; validators reading chain state agree on it by consensus.

In practice. A committer who wants to delay a reveal by ~1 hour sets `reveal_at = H_now + 300` (at 12-second blocks, ~3600 seconds). The reveal cannot land before block 300; honest validators reject earlier submissions. An adversarial validator who admits an early reveal produces a block other honest validators reject.

Reveal-after-TTL. Symmetric. A reveal submitted at $H_{\text{current}} \geq \text{reveal_at} + \text{ttl}$ is rejected; the commitment is in EXPIRED state, the runtime knows this from chain state.

Soft real-time only. “1 hour from now” is not exactly an hour; it’s 300 blocks at the chain’s nominal block time. If block time slows due to network conditions, the real-world time-lock duration grows. This is the intended semantics: chain height is the clock, not wall-clock time. Schemas requiring wall-clock-bounded windows must accept the approximation.

5.5 Hash Function Choice

The schema declares its hash function at registration (§3.4). Three currently-supported choices:

Hash	Output	Collision resistance	Pre-image	Notes
SHA-256	256 bits	2^{128}	2^{256}	Default; universally supported
BLAKE3	256 bits	2^{128}	2^{256}	~5x faster on modern CPUs
Poseidon	256 bits	$\sim 2^{128}$	$\sim 2^{256}$	ZK-friendly; slower verification

All three meet the protocol’s collision-and-pre-image-resistance requirements at the 128-bit security level. BLAKE3 is the speed recommendation for high-throughput auction schemas; Poseidon is the only option admitting efficient zero-knowledge reveal variants (§9.1).

Hash is per-schema, not per-commitment. A commitment’s hash is implicit from its schema’s declaration. There is no per-commitment hash field; this prevents an adversary from submitting a commitment under SHA-256 and trying to reveal under a weaker hash.

Migration. When (or if) any hash is broken cryptanalytically, schemas migrate by registering a new schema-id with a new hash. The §3.4 schema-versioning model (matching native-delegation §10.4 and per-schema-fees §9.5) supports backward-compatible cohorts. Pre-break commitments live out their TTL under the old hash; post-break commitments under the new schema. No retroactive invalidation.

6. Use Cases and the Validation Gate

6.1 The Use-Case-Validation Gate

The engineering work for time-locked attestations begins only when at least one design partner per use-case category (auction, embargo, regulatory time-lock) has submitted a concrete use-case description. Until that gate is satisfied, this paper is specification-only.

Three categories. Each unlocks a different surface of the mechanism:

- **Sealed-bid auction:** high-throughput, large-deposit, short-window. Drives the §4.4 deposit-on-commit and §4.5 front-running defense surfaces.
- **Embargoed announcement:** low-throughput, optional-deposit, medium-window. Drives the §3.2 hiding-property surface (the payload must remain hidden until reveal) and the §8.1 never-reveal failure-mode analysis.
- **Regulatory time-lock:** low-throughput, no-deposit, long-window. Drives the long-TTL handling (up to `t1_max` ~ 14 days) and the §8.2 late-reveal handling for partial-disclosure regimes.

A single use case per category satisfies the gate; subsequent partners can add their own cases within categories. The gate is a quality threshold, not a quota; it ensures the engineering investment is justified by demand across enough of the design space to validate the abstractions.

6.2 Use Case Template

Each design partner submits a description matching this template:

Field 1: Operator and workload. Who runs the workload? What's the expected per-day commit volume? What's the expected reveal-window distribution (mean, p99)?

Field 2: Payload structure. What does the committed payload look like? What's the payload entropy (binary, small-discrete, structured, free-form)? This drives the nonce-length requirement (§3.4).

Field 3: Deposit policy. Is there a per-commit deposit? At what level? Where does it go on cleanup (committer / treasury / burn / cleanup_runner / split)?

Field 4: Slashing requirements. Should never-revealed commitments slash the committer? At what magnitude? §8.1 names this as schema-author choice; the partner specifies what they need.

Field 5: Failure-mode requirements. Which §8 failure mode matters most? Never-reveal, late-reveal, front-running, hash collision, nonce reuse, reveal-DoS. Force a ranking; the engineering tightens the corresponding defense.

Field 6: Integration commitment. Does the partner agree to integrate against the v1.5 protocol during the early-stage pilot? With which timeline?

6.3 Hypothetical Use Cases (Not Yet Validated)

The following are illustrative, not validated. They are NOT commitments to ship until §6.2 descriptions are in hand.

Hypothetical 1: Sealed-bid NFT auction.

Operator: an NFT marketplace running sealed-bid auctions for limited drops. Volume: ~1000 commits per auction, ~5-second reveal window after close. Payload: bid amount (1-10000 range, 14 bits entropy) plus bidder address (32 bytes). Deposit: 10% of bid value (held to prevent commit-and-disappear). Failure-mode priority: front-running between commit and reveal (an adversary observing reveals in mempool would gain unfair information). Status: hypothetical; no marketplace partner has signed up.

Hypothetical 2: Embargoed press release.

Operator: a news outlet committing stories ahead of an embargo time (e.g., quarterly earnings disclosures, government announcements). Volume: 1-10 commits per day. Payload: structured press release (free-form text, low entropy in headline, high entropy overall). Deposit: zero (the operator's reputation is the stake). Failure-mode priority: hiding (the commit must not leak the headline before reveal). Status: hypothetical.

Hypothetical 3: Regulatory time-locked filing.

Operator: a financial filer submitting a regulatory disclosure that must land at deadline but become public only after a retention period (e.g., 30 days). Volume: ~10 commits per quarter. Payload: structured filing document (typically high entropy). Deposit: zero. Failure-mode priority: late-reveal (if the filer fails to reveal by `reveal_at + ttl`, the filing is considered missed and the filer faces regulatory penalties; this is application-layer, not chain-enforced). Status: hypothetical; pending regulatory-compliance partner conversation.

6.4 What §6 Looks Like at v0.2.x

When design partners submit §6.2 descriptions, §6.3 expands to include validated use cases with partner attribution, integration timelines, and any deviations from the v0.2 mechanism partners require. The §6.1 gate is satisfied; the engineering cycle for v1.5 time-locked attestations begins.

Until then, §6.3 hypotheticals are illustrative, not prescriptive. The paper is design-space documentation; the chain ships v1 without time-locked attestations.

7. Comparison with Prior Systems

Time-locked attestations occupy a distinct point in the design space against prior commit-reveal mechanisms.

Axis	Off-chain commit-reveal	On-chain auction smart contracts	Drand timelock / VDF	Embargoed storage (Filecoin / Arweave)	Time-locked attestations (this paper)
Where the time-lock lives	Trusted off-chain operator	Per-application smart contract	Cryptographic primitive (delay function)	Per-network retrieval policy	Chain runtime (height-based)
Trust model	Trusted operator (auctioneer)	Trusted contract code (audit-dependent)	Trustless (cryptographic)	Trusted network (storage providers)	Trustless under BFT (chain consensus)
Reveal mechanism	Operator publishes payload	Committer submits reveal transaction	Cryptographic decryption at time T	Retrieve from storage post-embargo	Committer submits reveal transaction
Cost per commit	Off-chain	Contract execution	Off-chain crypto + on-chain decrypt	Storage fee	Single chain tx (\$4.1)
Liveness requirement	Operator honest + alive	Contract + reveal-tx liveness	Drand or VDF time	Storage network liveness	Chain BFT liveness
Hash collision concern	N/A (centralized)	SHA-256 per contract	N/A	N/A	Per-schema choice (\$5.5)
Cascade integration	No	No	No	No	Yes (via CSC §3.4)
Production status	Live (manual auctions, off-chain protocols)	Live (Vickrey on Ethereum, multiple variants)	Drand live since 2020; VDFs research-grade	Filecoin live; Arweave embargo live	v1.5 specification (this paper)

The four comparators each solve a different subset. **Off-chain commit-reveal** is the simplest deployment but requires trusting the operator. **On-chain auction smart contracts** eliminate that trust by encoding the protocol in a contract; the cost is per-application implementation and audit burden. **Drand timelock encryption** and **VDFs** provide trustless time-locks via cryptographic primitives; the trade-off is that the time-lock is purely cryptographic, not tied to chain state or attester sets, so integration with PoUA reputation is application-layer. **Embargoed storage networks** address a related but distinct problem (delayed retrieval); the time-lock is at the storage layer, not at the attestation layer.

Time-locked attestations occupies a different point: the time-lock is a runtime primitive on Ligat Chain, enforced by validators reading chain height (no exotic cryptography), tied directly to the attestation/attester-set/PoUA-reputation stack the chain already supports. The trade-off is that we inherit the chain’s BFT liveness assumptions; under network partitions long enough to delay reveal beyond TTL, commitments expire (which is the desired behavior in most use cases, but bears stating).

The unique value-add is **native cascade with attestations**. None of the four comparators model what happens when a revealed attestation is later slashed: this paper integrates cleanly with cross-schema-composition’s §3.4 validity state machine, so a downstream attestation referencing a revealed time-locked attestation participates in the cascade defined there.

8. Failure Modes

We enumerate six failure modes and bound the damage of each. The bounds rely on the §4 mechanism, the §5 cryptographic security, and standard PoUA assumptions; this section does not introduce new defenses, only documents how the existing surface bounds each attack.

8.1 Never-Reveal

Setup. A committer submits `MsgCommit` and never reveals. The chain holds the commitment until `reveal_at + ttl`, then it transitions to `EXPIRED`. The payload never becomes public.

Damage bound. Three: (a) chain state holds the `EXPIRED` commitment until cleanup; (b) the schema’s downstream consumers cannot reference the attestation (no payload to reference); (c) if the schema has a deposit, the committer loses it (per `deposit_destination`).

Defense. TTL forces deterministic expiry. Cleanup is permissionless (§4.3): any operator submits `MsgCleanup` on expired commitments and earns a small fee. State accumulates only until cleanup runs; in practice, cleanup runs near-real-time because the cleanup-runner reward is non-zero.

Per-schema slashing. A schema with a high never-reveal cost (sealed-bid auction where unrevealed bids disrupt the auction) can declare a slash rule at registration: never-revealed commits over a threshold (say, >10% of committer’s commits in the prior 30-day window) trigger a PoUA-style reputation slash. The slash applies via PoUA §4.5, not as a new mechanism.

Economic incentive to reveal. Per §4.4, the deposit is held in escrow and returns on reveal. Auction schemas typically set `deposit_floor` to a meaningful fraction of bid value; the committer loses the deposit if they don’t reveal, which is sharply incentive-aligned.

8.2 Late-Reveal

Setup. A committer attempts `MsgReveal` after `reveal_at + ttl`. The chain is in `EXPIRED` state; the reveal admission check rejects.

Damage bound. None to the chain. The attempted reveal pays admission cost (small, bounded by mempool spam protection) and is rejected. From the application’s perspective, the committer “missed the deadline” and the schema’s downstream consumers proceed without the late attestation.

Application-layer recovery. Some schemas may allow a “late reveal that doesn’t count toward the original commitment” pattern, where the committer submits a fresh `MsgCommit + MsgReveal` with `reveal_at` in the past (so reveal is immediate). This is not a chain feature; it is application-layer behavior: the original commitment is gone, a new attestation under the same schema can land if the schema’s application logic accepts late attestations.

Pre-TTL late-reveal. A reveal after `reveal_at` but before `reveal_at + ttl` is *accepted* (this is the normal reveal window). Some application designs (auction with hard close) want a strict deadline at `reveal_at` itself, with no grace window; this is achieved by setting `ttl = 1` (reveal must land in the same block as `reveal_at`).

8.3 Front-Running Between Commit and Reveal

Setup. A reveal (payload, nonce) enters the mempool. An adversary observes it before block inclusion. The adversary submits their own `MsgCommit` with the now-known payload (or a derivative), hoping to commit-and-reveal in a way that beats the original committer.

Defense (§4.5 batched-reveal sequencing). All reveals in a block are processed before any new commits in the same block. An adversary cannot commit-then-reveal in the same block as another committer’s reveal, because the §4.5 ordering rule puts the reveal first. The adversary’s commit lands in a later block, by which time the original reveal is already canonical.

Defense (per-schema attestor set). Even if an adversary observes the payload, they cannot reveal it: the reveal carries the same threshold-signature from $\mathcal{A}_\sigma$ as the commit. Without the attestor-set’s signature, the adversary’s `MsgReveal` is rejected at admission.

What this defense leaves open. An adversary with the payload but not the attestor-set signature can attempt a *new* commitment under a different schema (or under a schema where they control the attestor set). They cannot interfere with the original commitment; they can attempt a parallel attack with their own commitment. Whether that succeeds depends on the application-layer’s response to multiple commits over the same underlying claim.

Bound. The chain prevents front-running of the original commitment with high confidence. Application-layer concerns about parallel commitments are outside the chain’s scope.

8.4 Hash Collisions

Setup. An adversary finds two reveals (payload₁, nonce₁) and (payload₂, nonce₂) with payload₁ ≠ payload₂ that both hash to the same commitment h . They commit, then reveal a different payload than they originally intended (e.g., bid amount low at commit, high at reveal if they “win” the auction).

Defense. §5.1 binding under SHA-256 collision-resistance. Constructing a collision requires $\sim 2^{128}$ work; far beyond practical attack budgets.

Defense if SHA-256 is broken. If a cryptanalytic break reduces SHA-256 below 2^{80} work, schemas migrate to BLAKE3 or Poseidon via the §3.4 hash-function migration path. The chain enforces the schema’s declared hash; per-commitment hash flexibility is not allowed (preventing the adversary from down-grading to a weaker hash).

Bound. Hash-collision attacks are infeasible at the 128-bit security level. The chain inherits hash-function security as an assumption; cryptanalytic monitoring is a community responsibility (not a chain mechanism).

8.5 Nonce Reuse

Setup. A committer reuses the same nonce across two different commitments with different payloads. An adversary observing both commitments cannot directly recover the payloads, but they have two hashes computed from the same nonce against two payloads: if the payload space is small enough, they can correlate brute-force attempts across both hashes, reducing the effective security.

Defense (per-commitment nonce derivation). §4.1 recommends nonces be derived as $\text{nonce} = H(\text{master_secret} \parallel \text{commitment_id}^*)$ where commitment_id^* is a deterministic counter (committer’s per-schema submission count). Each commitment gets a unique nonce by construction; reuse is impossible if the derivation is followed.

Bound under correct nonce-derivation. No degradation; nonce reuse cannot happen.

Bound under nonce reuse (committer error). If a committer reuses a nonce across N commitments, the effective security degrades from $2^{|\text{nonce}|}$ to $2^{|\text{nonce}|}/N$ (the adversary can amortize the brute-force across N commitments). At 128-bit nonces and 100 reuses, effective security is 2^{121} , still infeasible. At 64-bit nonces and 100 reuses, effective security is 2^{57} , brute-forceable on a small cluster. The 128-bit nonce floor (§3.4) is the protocol-level safety margin against this error.

Application-layer guidance. Schema implementations should derive nonces deterministically and never expose nonce-generation to user code. The chain does not enforce this; it is application discipline.

8.6 Reveal-DoS

Setup. An adversary submits a flood of `MsgCommit` transactions with no intent to reveal. Each commitment occupies chain state until `reveal_at + ttl`. If `ttl_max` is large (e.g., 14 days), the adversary can occupy substantial chain state for an extended period.

Defense (cleanup-runner incentive). §4.3’s cleanup-runner economics ensure that expired commitments are pruned in near-real-time. The cleanup-runner reward is funded from the schema’s per-commit fee, so even minimum-deposit commitments fund some cleanup.

Defense (deposit floor). A schema with high adversary exposure (auction schemas, regulatory schemas) declares a non-zero `deposit_floor` at registration. The adversary must lock up `deposit_floor` per commit; flooding becomes economically expensive.

Defense (per-schema per-account commit rate limit). Optional. A schema can declare a rate-limit at registration: maximum commits per account per epoch. Adversaries with many accounts circumvent per-account limits but at the cost of operating many addresses.

Defense (per-attestor-set spam protection). The threshold-signature requirement on commits means only the attestor set can submit. An adversary outside the attestor set cannot flood

commits at all. For schemas with small attester sets, this is the dominant defense; for schemas with permissionless or large attester sets, additional rate-limiting matters.

Bound. Combination of cleanup-runner economics + deposit floor + rate limits + attester-set bound makes reveal-DoS impractical at the protocol level. The residual risk is implementation: if a schema misconfigures (zero deposit, no rate limit, permissionless attester set), the schema author owns the consequence.

9. Limitations and Future Work

The v0.2 mechanism specifies hash-based commit-reveal under classical hash functions with chain-height time-locks. Four extensions remain out of scope; we document each here.

9.1 ZK-Friendly Variant

A SNARK-based reveal would let the chain verify “the committed payload satisfies predicate P” without learning the payload itself. Applications: sealed-bid auctions where the chain verifies “this bid is in valid range [min, max]” without disclosing the bid value, even on reveal; regulatory filings where the filer proves compliance without disclosing sensitive content. The §3.4 Poseidon hash option exists for this future variant; v0.2 specifies the cleartext-reveal mechanism only.

The complications are non-trivial. SNARK verification at admission time is expensive ($O(10^4)$ gas equivalents in modern proof systems); proof generation is slow ($O(\text{seconds})$ on commodity hardware for medium-complexity predicates). The runtime would need to bound proof-verification cost per `MsgReveal`. v0.3+ work; a follow-up paper.

9.2 Cross-Chain Time-Locks

A commitment on Ligate Chain whose reveal is gated by a foreign chain’s block height (or by Drand round, or by external clock) is the cross-chain time-lock problem. Mechanically: the reveal admission check requires an IBC light-client proof of the foreign chain’s height, plus the time-lock condition evaluates against that. Complications: IBC update latency means the foreign chain’s height-as-known-to-Ligate is stale by the round-trip; cascade interaction with cross-schema-composition’s §3.4 needs re-validation. Out of scope for v0.2; follow-up paper.

9.3 VDF-Based Hard Time-Locks

A VDF-based time-lock would provide “no early reveal even if the committer wants to”: the committer encrypts the payload such that decryption requires T sequential VDF steps, then commits the ciphertext. Even the committer cannot reveal before T elapses. Different security model: trustless time-lock (no chain consensus required for the time-lock itself), at the cost of VDF computation overhead. Not a v1.5 priority; positioned in §7 as a complement, not a competitor.

9.4 Multi-Party Commit-Reveal

An M-of-N reveal pattern (any M of N original committers can reveal) is useful for multi-party-controlled commitments (escrow agents, multi-stakeholder embargos). Requires additional cryptography: either (a) M-of-N threshold-signing the reveal (already supported via the attester-set threshold, but the original attester set may not be the M-of-N reveal set), or (b) Shamir-style

secret-sharing of the payload-and-nonce across the N parties. v0.2 specifies single-attestor-set-reveal; multi-party variants are follow-up.

10. Conclusion

Time-locked attestations as a runtime primitive eliminate the trusted auctioneer or coordinator from commit-reveal workflows. The §1.1 late-disclosure thesis is the motivation: some attestations should not be readable at submission time, and the chain should enforce that without an off-chain intermediary. The §1.2 off-chain trust problem is the central justification: every off-chain commit-reveal protocol carries availability + confidentiality + integrity + censorship-resistance assumptions that are hard to harden; an on-chain primitive replaces all four with chain BFT consensus.

The paper’s four contributions resolve the design space. (1) **Mechanism (§3 + §4)**: three transaction types over a four-state lifecycle, with optional deposit-on-commit and a permissionless cleanup market. (2) **Cryptographic security (§5)**: binding under collision-resistance, hiding under pre-image-resistance and 128-bit nonce floor, time-lock security under chain consensus. (3) **Failure-mode analysis (§8)**: six failure modes (never-reveal, late-reveal, front-running, hash collisions, nonce reuse, reveal-DoS) each bounded by structural and economic defenses. (4) **Use-case validation gate (§6)**: explicit framing that engineering work begins only when at least one design partner per category (auction, embargo, regulatory) submits a concrete use case.

The mechanism is positioned as a **v1.5 protocol feature**, post-devnet but pre-mainnet. Ligate Chain v1 ships with single-phase attestations; the four flagship products do not need commit-reveal at launch. Time-locked attestations land when §6.1 gate is satisfied. This paper documents the design space and security argument so that, when the gate opens, engineering work has a target to ship against.

What this paper does not do. It does not advocate for shipping time-locked attestations on v1 day one. It does not claim that workflows currently using single-phase attestations should switch. It does not commit Ligate Chain to a v1.5 release date.

What this paper does do. Capture the design space at the point in time when the trade-offs are fresh, the cryptographic argument is standard, and the integration with companion primitives (native delegation, per-schema fees, cross-schema composition) is concrete. If and when design-partner demand validates the use cases, the engineering cycle has a reference document.

Invitations. Paper, future simulator, and chain implementation are open to external review. The §6.2 use-case template is open to design-partner submissions through hello@ligate.io. Feedback on §5 cryptographic security (especially nonce-derivation patterns and hash-function migration timelines) is welcome from cryptography researchers.

The §1.4 central question was: what is the minimum on-chain commit-reveal primitive that handles auction / embargo / regulatory use cases with adequate cryptographic security and explicit never-reveal cleanup, without re-introducing off-chain trust assumptions? This paper answers: three transactions, four states, hash-based commitments under standard cryptographic assumptions, validator-enforced time-bounds, permissionless cleanup market. The mechanism is small, the argument is tight, the integration with adjacent primitives is orthogonal.

References

Cryptographic commitments and time-locks.

- Pedersen, T. (1991). *Non-interactive and information-theoretic secure verifiable secret sharing*. CRYPTO 1991.
- Lamport, L. (1979). *Constructing digital signatures from a one-way function*. SRI International CSL-98.
- Wesolowski, B. (2019). *Efficient verifiable delay functions*. EUROCRYPT 2019.
- Pietrzak, K. (2018). *Simple verifiable delay functions*. ITCS 2019.

Auction theory.

- Vickrey, W. (1961). *Counterspeculation, auctions, and competitive sealed tenders*. Journal of Finance, 16(1).
- Clarke, E. (1971). *Multipart pricing of public goods*. Public Choice, 11.
- Groves, T. (1973). *Incentives in teams*. Econometrica, 41(4).

On-chain randomness and commit-reveal protocols.

- Drand (2017+). *Distributed randomness beacon*. <https://drand.love/>
- Galindo, D., Liu, J., Ordean, M., Wong, J. (2021). *Fully distributed verifiable random functions and their application to decentralised random beacons*. EuroS&P 2021.
- RANDAO (Ethereum beacon chain randomness). <https://eth2book.info/altair/part3/transition/epoch>

Hash functions.

- NIST (2002). *FIPS 180-2: Secure Hash Standard (SHS)*. (SHA-256 family.)
- Aumasson, J., Neves, S., Wilcox-O’Hearn, Z., Winnerlein, C. (2020). *BLAKE3: One Function, Fast Everywhere*. <https://github.com/BLAKE3-team/BLAKE3-specs>
- Grassi, L., Khovratovich, D., Rechberger, C., Roy, A., Schafneggger, M. (2021). *Poseidon: A New Hash Function for Zero-Knowledge Proof Systems*. USENIX Security 2021.

Companion Ligate Labs research.

- Ligate Labs (2026). *Proof of Useful Attestation*. Working paper v0.8.
- Ligate Labs (2026). *Native Delegation as a Runtime Primitive*. Working paper v0.2.
- Ligate Labs (2026). *Per-Schema Fee Markets*. Working paper v0.2.
- Ligate Labs (2026). *Cross-Schema Composition*. Working paper v0.2.
- Ligate Labs (2026). *Schema-Bound Tokens*. Working paper v0.1.

Chain stack.

- Sovereign Labs (2024). *Sovereign SDK*. <https://github.com/Sovereign-Labs/sovereign-sdk>
- Celestia Labs (2023). *Celestia: Modular Data Availability*. <https://celestia.org/learn/>

Appendix A: Simulator Validation Plan

The reference simulator at `prototypes/time-locked-attestations-sim/` shipped with v0.2.1 (M1 milestone). It validates the cryptographic and mechanism layers of the paper. The §6 use-case-

validated work (strategic-bidder sealed-bid auction game, embargo-leak game) is explicitly **out of M1 scope** and lands when the §6.1 design-partner gate opens.

Shipped modules under `src/time_locked_attestations_sim/`:

- `commitment.py`: §3.1 Commitment tuple, §3.4 hash-function dispatch (SHA-256 default, BLAKE3 with optional native dispatch, Poseidon signaled-but-not-implemented per §9.1).
- `lifecycle.py`: §3.3 four-state machine (COMMITTED, REVEALED, EXPIRED, CLEANED-UP) with deterministic transition rules.
- `transactions.py`: §4.1 / §4.2 / §4.3 admission-time checks for `MsgCommit`, `MsgReveal`, `MsgCleanup`, plus §4.5 batched-reveal sequencing.
- `failure_modes.py`: §8 attack-scenario harnesses (never-reveal, late-reveal, front-running, hash collisions, nonce reuse, reveal-DoS) returning structured `FailureModeResult` records with quantitative bounds.

Shipped test coverage (41 tests, all passing):

- §3.3 lifecycle state-machine transitions, including terminal-state persistence
- §3.4 nonce-floor enforcement (rejects nonces below 128 bits)
- §4.1 / §4.2 / §4.3 admission-time checks with tagged rejection reasons for mempool diagnostics
- §4.5 block-sequencing rule (reveals before cleanups before everything else)
- §5.1 binding via collision-distinct-input test
- §5.3 nonce-entropy bound: 128-bit floor keeps even $N = 1000$ reuses at 118 bits effective security
- §5.4 time-lock via reveal-before-`reveal_at` and reveal-after-TTL both rejected
- §8.1 never-reveal: deterministic transition to EXPIRED
- §8.5 nonce-reuse: quantitative bit-security degradation as a function of N
- §8.6 reveal-DoS: economic-bound assertion under deposit and cleanup pricing

Cross-language test vectors at `test_vectors/commitment_canonical_encoding.json` carry four canonical SHA-256 commitment hashes (binary YES/NO, sealed-bid integer, long embargoed text) for downstream Rust / TypeScript verification. Format matches the convention of `per-schema-fees-sim/test_vectors/` and `native-delegation-sim/test_vectors/`.

Headline figure `out/failure_modes_panel.png`: a 2×3 panel summarizing each §8 failure mode’s quantitative bound. Generated by `scripts/run_failure_mode_panel.py`.

What M1 does not cover. The simulator stops at the cryptographic and mechanism layers. Strategic-bidder game-theory simulation (sealed-bid auction with N agents, valuation distributions, reveal-or-default equilibrium) is M2 territory and gated on §6.1 design-partner validation. The decision to ship M1 ahead of the §6 gate reflects that the cryptographic and mechanism claims are partner-independent; the auction and embargo game theory is not.

Appendix B: Formal Definitions

We collect the formal definitions used throughout the paper in one place.

Definition (Commitment). A tuple $c = (h, \text{reveal_at}, \text{ttl}, \sigma, \mathcal{A}_\sigma, d)$ where $h = H(\text{payload} \parallel \text{nonce})$ is the commitment hash, `reveal_at` is the earliest reveal block, `ttl` is the reveal-window length, σ is the schema-id, $\mathcal{A}_\sigma$ is the attester set, and d is the optional deposit.

Definition (Reveal). A pair (payload, nonce) submitted within the reveal window (block height in the half-open interval from `reveal_at` to `reveal_at + ttl`), where $H(\text{payload} \parallel \text{nonce})$ matches the committed hash and the submission carries the same threshold signature as the original commitment.

Definition (Validity state). For each commitment c , $\text{state}(c)$ is one of: COMMITTED, REVEALED, EXPIRED, CLEANED-UP. Transitions per §3.3.

Definition (Cleanup-runner reward). A small protocol fee rebated from the schema’s per-commit fee to the address submitting a successful `MsgCleanup` on an EXPIRED commitment. Funds the permissionless cleanup market.

Definition (Binding). Under the chosen hash function H , the commitment scheme is binding if no efficient adversary can produce two reveals $(\text{payload}_1, \text{nonce}_1) \neq (\text{payload}_2, \text{nonce}_2)$ that both pass the reveal check against the same commitment h . Proven for SHA-256, BLAKE3, Poseidon at the 128-bit security level per §5.1.

Definition (Hiding). Under the chosen hash function and nonce length, the commitment is hiding if no efficient adversary can recover payload from h alone with probability significantly better than $|\text{payload-space}| \cdot 2^{-|\text{nonce}|}$. Proven for SHA-256 / BLAKE3 / Poseidon with the 128-bit nonce floor per §5.2.

Definition (Time-lock security). Under chain BFT liveness, a reveal is admitted if and only if the inclusion height lies within the reveal window (the half-open interval starting at `reveal_at` and ending at `reveal_at + ttl`). The chain’s height is the canonical clock; no exotic cryptography (VDF, time-lock encryption) is required.